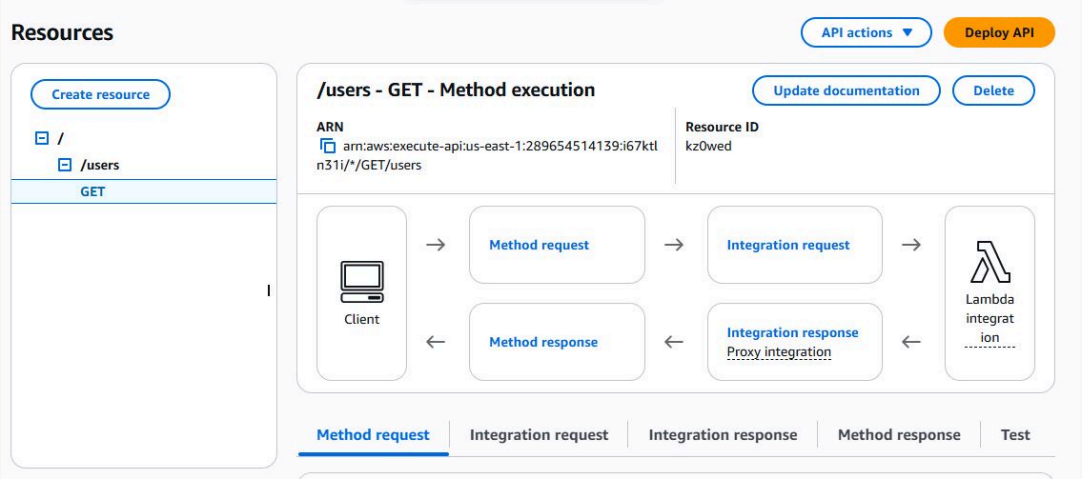
nextwork.org

APIs with Lambda + API Gateway

Mohammed Dawoud Mota



Introducing Today's Project!

Today I’m here to walk through the process of building a serverless API using AWS Lambda and Amazon API Gateway. My goal in this project is to understand how backend logic can run without managing any servers and how API Gateway acts as the entry point that allows users to interact with that logic. By completing this work, I’m learning how to create and configure a Lambda function, set up an API in API Gateway, connect the two together, and deploy the API so it becomes accessible through a public endpoint. This project also fits into a larger three tier architecture series, and in this part I’m focusing specifically on the logic tier, where the backend processing of an application takes place.

Tools and concepts

In this project I learned how several core AWS services work together to create a functioning backend. I worked with AWS Lambda to run my code without managing servers, and I used API Gateway to create a structured API with resources, methods, and a public endpoint. I also learned how deployment stages work and how they allow me to publish different versions of my API. Altogether these concepts helped me understand how serverless applications are built, organized, and made accessible through the cloud.

Project reflection

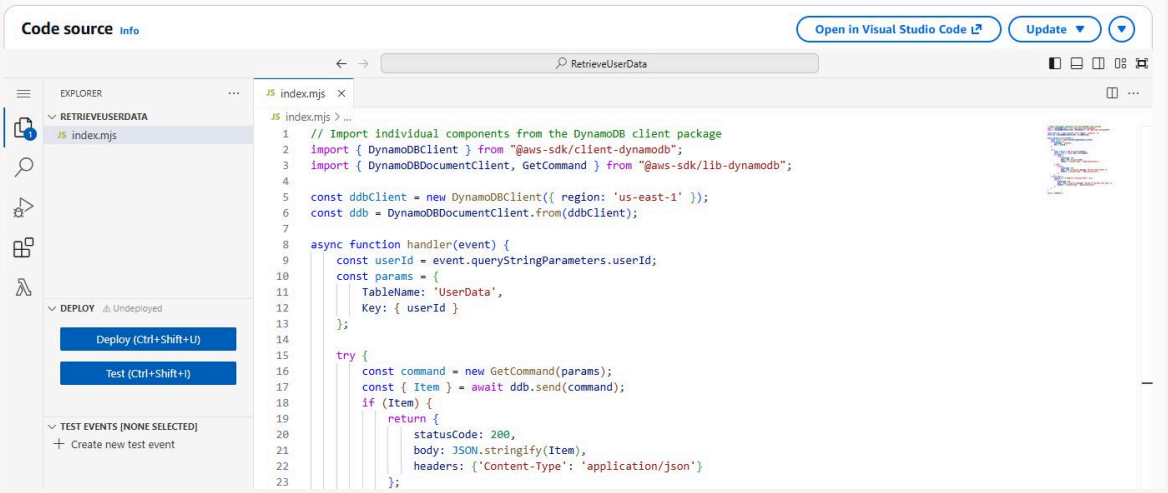
It took me only a short amount of time to complete this project because each step built directly on the one before it. Once I created the Lambda function and set up the API structure, the remaining tasks like adding the resource, creating the method, and deploying to a stage moved quickly. Overall the project was straightforward, and going through it step by step made the entire process smooth from start to finish.

I chose to do this project today because... Something that would make learning with NextWork even better is...

Lambda functions

AWS Lambda is a service that lets me run code without having to manage any servers myself. Instead of setting up or maintaining any infrastructure, Lambda handles all of that behind the scenes and only runs my code when it is needed. This means I am not paying for idle time, and my function can scale automatically from just a few requests to thousands per second. In this project, I am using Lambda to power the backend logic of my API, allowing it to retrieve user data and return responses efficiently as part of the serverless architecture I am building.

The code I added to my Lambda function is responsible for retrieving user data from a DynamoDB table based on a userId that comes in through the API request. It creates a DynamoDB client, reads the userId from the query string parameters, and then uses that value to look up a matching item in the UserData table. If the item exists, the function returns it with a successful status code. If no matching record is found, it returns a message indicating that no user data exists. If something goes wrong during the process, the function returns an error response. Overall, this code prepares the Lambda function to act as the backend logic that will eventually power the API once the database is set up.

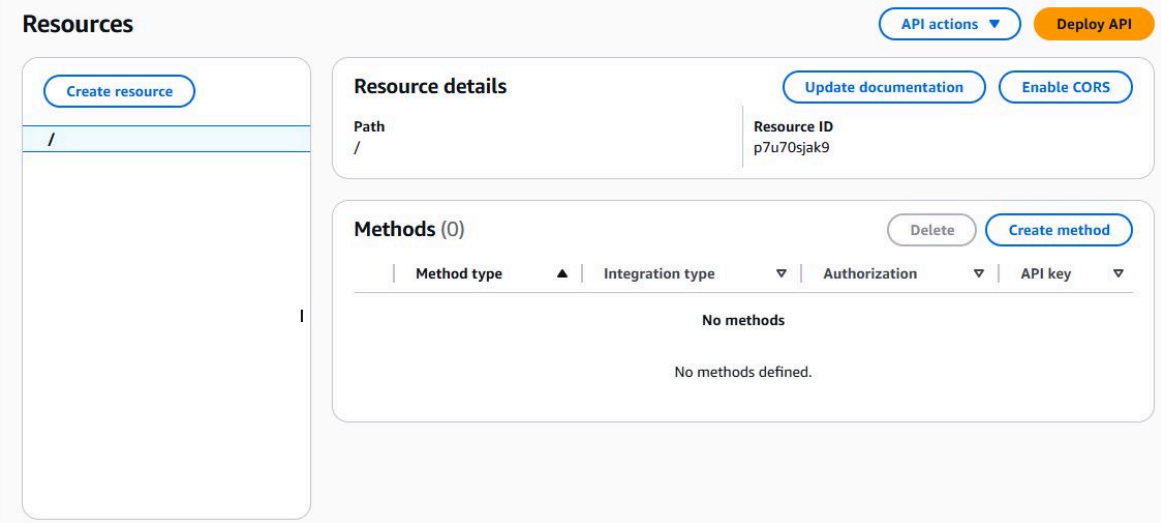

Code source Info
Open in Visual Studio Code
Update
RetrieveUserData
EXPLORER
RETRIEVEUSERDATA
index.mjs
DEPLOY
Undeployed
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
TEST EVENTS [NONE SELECTED]
Create new test event
// Import individual components from the DynamoDB client package
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
const ddb = DynamoDBDocumentClient.from(ddbClient);
async function handler(event) {
const userId = event.queryStringParameters.userId;
const params = {
TableName: 'UserData',
Key: { userId }
};
try {
const command = new GetCommand(params);
const { Item } = await ddb.send(command);
if (Item) {
return {
statusCode: 200,
body: JSON.stringify(Item),
headers: {'Content-Type': 'application/json'}
};

API Gateway

An API, or Application Programming Interface, is a structured way for different software systems to communicate with one another. It acts like a messenger that carries a request from one system to another and then returns the response back to the requester. In the context of this project, my API is the layer that allows a user's browser to send a request that eventually reaches my Lambda function. The API handles the communication flow so the user can interact with the backend logic without ever seeing the underlying code or infrastructure.

API Gateway acts as the front door to my Lambda function by receiving incoming requests from a user and securely passing them to the backend for processing. It handles important responsibilities like routing traffic, managing access, and ensuring that only authorized requests reach my Lambda function. I am using API Gateway in this project because Lambda on its own is not designed to be exposed directly to the public, so API Gateway provides the structure, security, and management needed to make my serverless API work smoothly and reliably.

Lambda and API Gateway work together by forming a clean, serverless pipeline between the user and the backend logic. API Gateway receives the incoming request from the browser and acts as the controlled entry point, handling things like routing, security, and traffic management. Once the request reaches API Gateway, it forwards the full request to my Lambda function, which processes the input, runs the backend code, and prepares a response. After Lambda finishes, the response is sent back through API Gateway, which then returns it to the user. This partnership allows me to build an API without managing any servers while still keeping the flow organized, secure, and efficient.

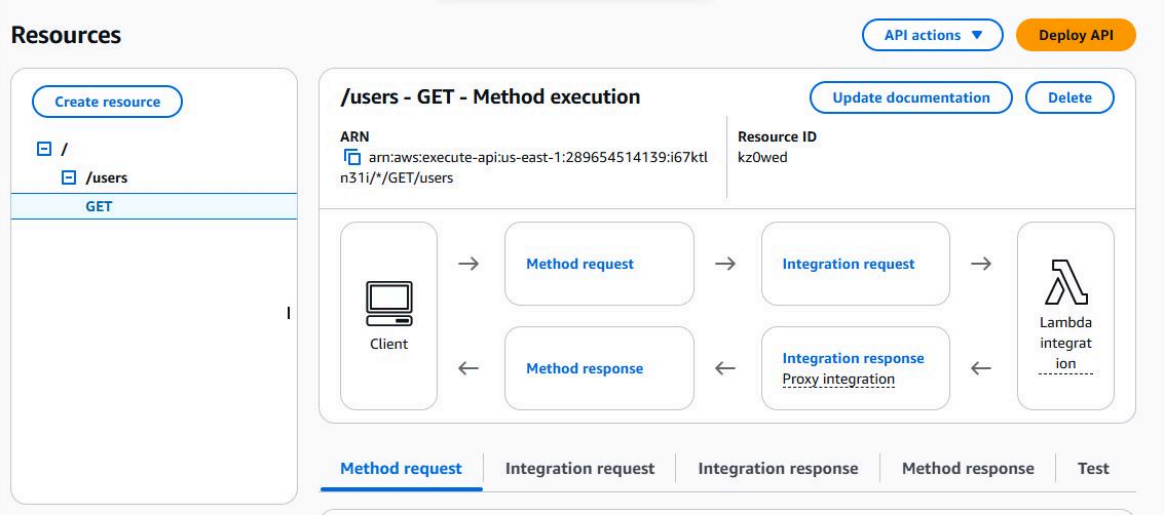

Resources
API actions
Deploy API
Create resource
/
Resource details
Update documentation
Enable CORS
Path
/
Resource ID
p7u70sjak9
Methods (0)
Delete
Create method
Method type
Integration type
Authorization
API key
No methods
No methods defined.

API Resources and Methods

An API resource represents a specific section or category within an API that groups related functionality under a clear path. It helps organize the API so different types of requests have their own dedicated place to go. For example, creating a resource called users gives me a structured endpoint where all user-related actions can be handled. This makes the API easier to manage, easier to understand, and more aligned with how real applications separate different parts of their functionality.

Methods are the actions an API can perform on a specific resource, and they define how that resource behaves when someone interacts with it. Each method corresponds to a standard HTTP action such as GET for retrieving data, POST for creating new data, PUT for updating existing data, and DELETE for removing data. By adding a method to a resource, I am telling the API exactly what kind of operation should happen when a user makes a request to that part of the API.

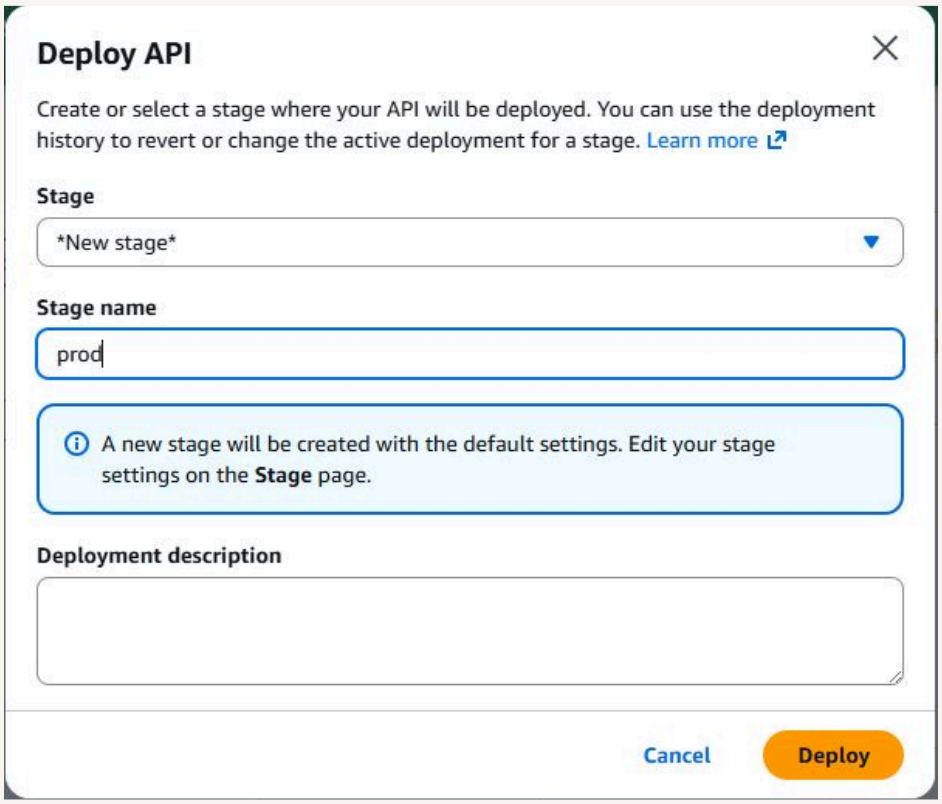
The method I set up is a GET method for the users resource. This method allows the API to retrieve user information when someone makes a request to that specific endpoint. By choosing GET, I am defining that this part of the API is meant for reading data rather than creating, updating, or deleting anything. This method is then connected to my Lambda function so that whenever the GET request is made, the Lambda function runs and processes the request.


Resources
API actions
Deploy API
Create resource
/
/users
GET
/users - GET - Method execution
Update documentation
Delete
ARN
arn:aws:execute-api:us-east-1:289654514139:i67ktln31i/*/GET/users
Resource ID
kz0wed
Client
Method request
Integration request
Lambda integration
Method response
Integration response
Proxy integration
Method request
Integration request
Integration response
Method response
Test

API Deployment

A stage is the environment where my API becomes live after I deploy it. When I publish everything to a stage like prod, I am taking all the work I have done and making it available through a real URL that I can use for testing or for actual users. Stages also help me keep different versions of my API organized, so I can have one version for development and another one for production without affecting each other.

I went to the invoke URL that API Gateway gave me after I deployed the API to the prod stage. That URL is the public endpoint that represents the live version of my API, so visiting it in the browser lets me see the actual response coming from my Lambda function. It is the place where everything I built becomes accessible, and it confirms that the API, the method, and the Lambda integration are all working together the way they should.



The place to learn & showcase your skills

Check out nextwork.org for more projects